Decision Orchestration: Learning Annotation Procedures from a Handful of Gold Annotations

Anonymous ACL submission

Abstract

A new annotation scheme, language or domain usually starts with a guideline and little gold data. Prompting an LLM for the whole annotation can violate the scheme's conventions and produce malformed output, while supervised systems need thousands of examples. We observe that a guideline is mostly a procedure: most steps are deterministic, and only some call for judgment. We propose *decision orchestration*, which encodes a guideline as an executable procedure separating rule nodes from model decision nodes. Code runs the steps and assembles the output; a model is consulted only at decision points, for a schema-constrained fragment or for probabilities over the options of a closed question. We then learn the procedure itself, not model weights, from a handful of gold annotations: thresholds are tuned against the task metric, uncertain decisions are routed to a stronger model, and an LLM reflector proposes edits that, for closed decisions, are kept only if they measurably improve the score. On verbal multiword expression identification (PARSEME), a closed procedure learned from 60 gold sentences, five at a time, reaches 46.3 F in English, 4.4 points above the best published result we know of and 13 above the best system of the 2018 shared task, and 57.8 F in Chinese, 18 points above an LLM annotating directly with ten gold examples; reflection costs under one dollar per language. On Uniform Meaning Representation parsing, where the output is mostly graph structure, a 9B model inside the procedure reaches 17.6 F against 4.9 for a detailed prompt, two gold documents of reflection raise a frontier model from 45.3 to 52.6 F, and routing adds 2.5 F to a closed procedure; yet direct generation with ten gold examples matches the best closed procedure in Chinese and exceeds it in English. Decision orchestration pays off most where a guideline is a sequence of tests over enumerable candidates.

1 Introduction

New annotation schemes, and old schemes in new languages or domains, often start with a prose guideline and little gold data. Producing the first annotations is costly (Settles, 2009). LLMs can assist familiar labeling tasks (Gilardi et al., 2023; Tan et al., 2024), but whole-structure prompting can violate a scheme's conventions and produce malformed output (Ettinger et al., 2023; Sun et al., 2024), and supervised systems need thousands of annotated sentences.

We propose **decision orchestration** (Figure 1): manually encode a guideline as an executable procedure, separating deterministic dispatch and assembly from local model judgments. Code enumerates candidates, looks up resources and writes the output; models decide what needs understanding, such as a verb's sense or whether a verb and a noun form a light-verb construction. A decision node returns a schema-constrained fragment or distributions over closed options enumerated by code, so LLMs and closed-question services (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026) can serve the same procedure. The procedure is then adapted under gold supervision without updating model weights: **threshold tuning** fits how rules read answer probabilities to the task metric, **routing** sends uncertain answers to a stronger model, and **reflection** proposes local edits from aligned traces, which in closed mode are kept only if they measurably help, with one gold document or five gold sentences per round. Every revision is a readable artifact.

We test the method on two schemes that differ in how much of the output is structure, each in Chinese and English: verbal multiword expressions (PARSEME; Ramisch et al., 2018, 2020), sets of categorized token groups, and Uniform Meaning Representation (UMR; Van Gysel et al., 2021), graphs. Our findings:

- On PARSEME, a closed procedure learned from

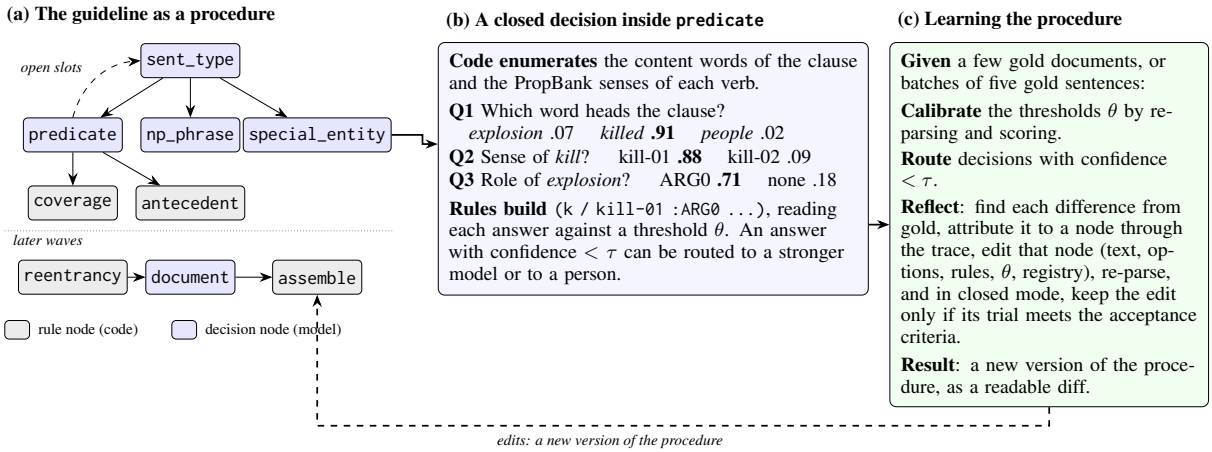


Figure 1: Decision orchestration. (a) The guideline is compiled into skills dispatched by a registry. Rule nodes are code; decision nodes consult a model. Shown is part of the UMR procedure (all nodes: Table 10, appendix); the PARSEME procedure is one rule node that enumerates candidates, one closed decision node and an output handler (Appendix H). (b) In closed mode, a decision node asks questions whose options code has enumerated, and rules read the answer probabilities. (c) With a handful of gold annotations, the procedure itself is learned. Example sentence, in English for readability: *Police said the explosion killed two people.*

60 gold sentences, five at a time, reaches 46.3 F in English, **4.4 points above the best published result** we know of (Rohanian et al., 2019) and 13 above the best 2018 shared-task system, both trained on 3,471 sentences, and 57.8 F in Chinese, **18 points above** Claude Haiku annotating directly with ten gold examples, for under a dollar of reflection per language.

- On UMR, a 9B model reaches 17.6 F inside the procedure against 4.9 with a detailed prompt, two documents of reflection raise Sonnet from 45.3 to 52.6 F, and routing adds 2.5 F to a closed procedure. But direct generation with ten gold examples matches the best closed procedure in Chinese (42.0 vs. 42.6) and exceeds it by 16 F in English: where the output is mostly structure, closed decisions are not enough.
- Measurement shows where a procedure departs from the annotation: offering all 27 aspect values of the UMR guidelines, where the gold uses seven, together with one dropped polarity rule, cost 5.2 F.

Scope and shared infrastructure. The companion study *Can LLMs Build Better Annotators? Self-Improving Harnesses for Ontology-Guided Annotation* examines instruction revision in a three-call whole-graph harness. It shares our UMR data, alignment and judge and reuses procedural runs for comparison; shared runs are not independent replication. The PARSEME and English UMR runs are ours alone.

2 Related Work

LLMs as annotators. LLMs can assist crowd labeling (Wang et al., 2021; Gilardi et al., 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024). Human–LLM collaboration allocates uncertain items or verifies labels (Li et al., 2023; Wang et al., 2024); we route decisions within structured items. GoLLIE (Sainz et al., 2024) learns to follow code-form guidelines, Code4Struct (Wang et al., 2023) represents event structures as code, and guideline learning (Pang et al., 2023) induces text from errors. Here the harness executes a manually encoded procedure whose rules and thresholds can also change.

LM programs and structured output. Prompt chains and decomposed prompting compose model calls (Wu et al., 2022; Khot et al., 2023); programs can call models or declare their constraints (Gao et al., 2023; Gupta and Kembhavi, 2023; Beurer-Kellner et al., 2023; Khattab et al., 2024). Grammar constraints control output form (Willard and Louf, 2023; Geng et al., 2023), potentially affecting reasoning (Tam et al., 2024). Unlike model-directed agent control (Yao et al., 2023), our registry dispatches typed emits, keeps rule nodes explicit and allows closed distributions as an alternative to fragments.

Optimizing prompts and programs. Prompts can be searched or optimized from feedback (Zhou et al., 2023; Pryzant et al., 2023; Yuksekgonul et al., 2025; Yang et al., 2024; Opsahl-Ong et al., 2024). GEPA reflects on traces (Agrawal et al., 2026); ACE grows a playbook (Zhang et al., 2026). Workflows and scaffolds can also be searched (Zhang et al., 2025; Hu et al., 2025; Zelikman et al., 2024). Self-feedback alone is unreliable (Huang et al., 2024). Our supervised reflection uses structured gold alignment to propose node-local edits to options, rules and text, with measured selection in closed mode. This resembles transformation-based learning and data programming in producing readable artifacts (Brill, 1995; Ratner et al., 2017), without claiming causal attribution from traces alone.

Threshold adaptation and cascades. Probability calibration and selective prediction study confidence and abstention (Guo et al., 2017; Jiang et al., 2021; Zhao et al., 2021; Kamath et al., 2020); cascades escalate to costlier models (Chen et al., 2024). We instead tune program thresholds against graph scores and escalate individual closed questions. Teacher agreement is reported separately from gold correctness.

The two tasks. Verbal multiword expression identification has been evaluated in the PARSEME shared tasks (Ramisch et al., 2018, 2020). Their systems are mostly taggers or parsers trained on the full training data, from 2020 on pretrained encoders, and later work on English 1.1 reached 41.9 F (Rohanian et al., 2019); edition 2.0 also reports instruction-tuned LLMs (Scholivet et al., 2026). Supervised AMR parsers likewise use large labeled corpora (Bevilacqua et al., 2021; Zhou et al., 2021; Bai et al., 2022; Knight et al., 2020); UMR has less gold (Van Gysel et al., 2021; Bonn et al., 2023, 2024, 2025). Pipeline conversion (Chun and Xue, 2024) and SETUP (Markle et al., 2025) are parsing alternatives; matched comparisons remain pending.

3 Decision Orchestration

Setting. An annotation scheme defines a structured annotation y for a sentence or document x: in PARSEME, a set of categorized, possibly discontinuous token groups per sentence; in UMR, one rooted graph per sentence plus document-level temporal, modal and coreference relations. In the cold start we have a guideline, the resources it refers to (for UMR, PropBank frames; Palmer et al., 2005), a scorer and a handful of gold annotations. We build, and later learn, a procedure Π that maps x to y.

Skills, registry and execution. Π consists of **skills**, a **registry** and numeric parameters θ. A skill has an input schema, an output schema (its *emit*) and an implementation: code (a *rule node*) or a model call (a *decision node*). The registry declares *waves* that run in order; within a wave, a dispatcher maps fields of a skill's emit to the next skill, growing a tree of nodes until every branch ends in a leaf. Control flow is thus a fixed function of typed decisions: answers can change tree shape and call counts, but the model cannot choose an undeclared skill. In UMR (Table 10, appendix), `sent_type` routes a span to `discourse`, `predicate`, `np_phrase` or `special_entity`, or stops it as a leaf; fragments leave typed placeholders that go back to `sent_type`; rule nodes redispatch uncovered tokens; later waves merge coreferring nodes and add document-level relations. The PARSEME procedure is the smallest instance: a rule node enumerates candidates from the tokens, lemmas, parts of speech and dependency relations supplied with the data, a closed decision node asks one question per candidate, and the output handler resolves overlaps and writes the `.cupt` file (Appendix H).

Two kinds of decision nodes. A decision node has two implementations that produce the *same* emit, so they can be swapped node by node. In a **fragment decision**, an LLM receives the skill's instructions (the relevant part of the guideline), the span and the sentence, may call tools for examples and frames, and answers through the emit schema, for example with a graph fragment; it never writes a whole graph. In a **closed decision**, the skill's code enumerates candidates from the input (the words of a span, the senses and roles of a verb, the allowed categories) and asks *closed questions*: yes/no, a choice among options that each carry a one-line description from the guideline, or a graded score. A decision model returns a distribution over the options and no text. The skill is a coroutine that yields rounds of questions and returns an emit built by rules, which read the distributions against thresholds θ. `predicate`, for instance, asks for the head, then its sense, aspect, modal strength and polarity, then the role of every content word (Figure 1b); a PARSEME candidate's question lists

the categories its kind allows plus *none* ("*gave + up*: VPC.full, VPC.semi, none"), and the candidate is kept when its best option is a category with $p \geq 0.5$. Closed mode needs no model that can write structured text, and every decision comes with a probability.

Traces and attribution. Every node records its input, its decision (in closed mode, the questions and full distributions) and its emit. The scorer's alignment of predicted to gold units lets us record the node that produced each aligned predicted unit; gold units that nothing predicted have no producer, such as a PARSEME expression that no rule proposed. This is output provenance, not causal attribution: upstream routing, candidate omissions and alignment errors can change downstream losses. A fine-grained UMR judge scores 25 aspects separately (Appendix B). Reflection reads these records.

4 Learning the Procedure from a Handful of Gold Annotations

All learning here changes Π (text, code, thresholds, registry) and never a model. For UMR documents D we measure

$$J(\Pi; D) = \frac{1}{|D|} \sum_{d \in D} \tfrac{1}{2}\big(F_d + C_d\big), \qquad (1)$$

where F_d is the sentence-graph F-score and C_d the comprehensive score that also covers the document level (Sun and Xue, 2024); in batch mode the score is the task's main metric on a set of sentences. Errors are diagnosed on *training* data; *regression* data (earlier training data) must not get worse; *development* data are measured but never shown for diagnosis, though acceptance uses their scores, so they are model-selection data. Test data are excluded from edit generation and acceptance.

Threshold tuning. Rules read probabilities against thresholds set for one decision model; another model spreads its probability mass differently, so a rule fires too often or too rarely. What matters is the decision a rule takes (cf. Zhao et al., 2021), so we fit θ to J by coordinate descent, sweeping one threshold at a time over a few values and keeping a value that raises J by more than 0.05.

Routing. A closed answer has a confidence $c = \max_o p(o)$. Questions with $c < \tau$ are asked again, with the same state and options, to a stronger answerer, an LLM forced to pick an option or, in a cold start, a person [E5]; nothing else changes.

Reflection. A round on a training document runs four steps with a strong LLM as the *reflector*. **Find**: from the gold and predicted annotations, the alignment, the judge's failed units and the trace, list each difference, the responsible node and the kind of cause. **Diagnose**: propose ranked changes per place in the procedure. **Edit**: write anchored edits. **Accept**: apply them to a copy of Π, the next frozen version. In fragment mode a parse takes minutes and dollars, so acceptance is *unmeasured*: an edit is accepted if its anchor matches once and the skill still loads, and the next document tests the round. In closed mode a parse takes seconds, so acceptance is *measured*: proposals are applied one at a time and the training, regression and development documents re-parsed after each. An edit is kept if J rises by at least 0.1, mean F and C fall by at most 0.3, no document falls by more than $\max(0.8, 2\Delta J)$ (capped at 2.5), and the regression documents fall by at most 0.3; a rejected edit is sent back once with its measurement.

Measured reflection in small batches. A document per round gives the reflector hundreds of problems at once and few rounds before the gold is used up. For the new runs we reflect on five gold sentences per round (Algorithm 1, appendix). The reflector reads, for each sentence, the gold and predicted annotations, the scorer's findings and every question asked with its top options and probabilities, together with the editable files and the edits tried before, and proposes at most three edits, each an exact find-and-replace in one file. Each edit is measured on the batch, a regression set (the two previous batches) and a fixed development set, and kept if the mean score rises by at least 0.3, the development score falls by at most 0.5 and the regression score by at most 1.0. Rejected edits go back once with their measured effect. Decisions are cached, so a trial pays only for the questions an edit changes.

Guards. The reflector never sees test data, and test scores come only from frozen versions. Batch-mode edits are checked for text copied from the test sentences (six Chinese characters or four English words in a row; none found). The closed document-mode check covers six-character Chinese substrings of a configured corpus subset, an

indirect use of test text whose coverage remains to be audited [E13]; in fragment mode it was a convention [TODO: run the check over the fragment-mode versions].

5 Experimental Setup

Benchmark 1: verbal multiword expressions. PARSEME annotates verbal multiword expressions (VMWEs): possibly discontinuous token sets with a category such as verbal idiom, light-verb or verb–particle construction (Ramisch et al., 2018, 2020). A Chinese VMWE is often one token, such as a verb with its complement (推出) or a four-character idiom. We use Chinese from edition 1.2 (train 35,326 sentences; test 3,462 sentences, 786 VMWEs) and English from edition 1.1, the last with English (train 3,471; test 3,965 sentences, 501 VMWEs). The procedure reads the lemmas, parts of speech and dependency relations supplied with the data. Batches come from the training file (three sentences with a VMWE, two without); the development set is 150 sentences.

Benchmark 2: Uniform Meaning Representation. UMR (Van Gysel et al., 2021; Bonn et al., 2025) represents a sentence as a rooted graph of concepts linked by roles, with attributes such as aspect and polarity, plus document-level relations. For Chinese we use the 31 gold documents available to us (712 sentences), preprocessed as in Chun and Xue (2024), with training documents from 0001–0014 and development documents from 0027–0029. Experiments were run in stages, so there are three test sets: T1 = 0004–0005 (99 sentences) and T2 = 0005–0007 (157) for fragment mode, and H = 0015–0018, 0021–0024 (138) for closed mode. H is excluded from reflection and tuning, but the authors have inspected it across versions, so comparisons on H are retrospective [E1]. For English we use the native annotation of UMR 2.0, with batches from the guideline's examples and a narrative, 25 development sentences, and 143 test sentences from news and narrative text, not used before the final runs. Appendix A lists the gold each version has seen.

Procedures. R_0 is the hand-written fragment seed and R_{xy} the version after round y on training document x. The closed seed J_0 was tuned by hand on 0001–0003; J_2 follows unmeasured reflection on 0001–0002, and K_1–K_4 continue from $K_0 = J_2$ with measured reflection on 0008–0011. In the batch runs, Π_r is the version after r rounds. The PARSEME seeds were written before any training sentence was read (Appendix H). For UMR we run batches from J_0 and from a revision of J_2 whose questions moved into data files and whose inventories were completed, including all 27 aspect values; its Π_0 restricts aspect to the seven values of the training gold and writes event negation as `:polarity -` (§7). English starts from the Chinese Π_0.

Models and baselines. Fragment decisions are made by Claude Sonnet 5.5, DeepSeek V4.1 Flash, Qwen3.5-9B and Qwen3.8-27B (Yang et al., 2025); closed decisions by Jev (TypeSafe, 2026), the OpenAI Decisions API (OpenAI, 2026) and Microsoft-Decision-1 (Microsoft, 2026), which share one protocol. The document-mode reflector is Claude Opus 5.5; the batch and routing runs use Jev 1.13, Claude Sonnet 5.5 as reflector and Claude Haiku 5.5 for routing. Baselines are a *single call* per sentence with a pointer to the guidelines, a *detailed single call* describing the target format, and *inventory + ten gold sentences*, in which Haiku writes the annotation given the category or concept inventory and the first ten pool sentences with their gold, a sixth of what batch reflection reads. For PARSEME we add the shared-task systems, rescored from their released predictions, and the best published English result we know of (Rohanian et al., 2019). Further baselines are pending [E2].

Metrics. PARSEME: the official MWE-based F1 (exact tokens and category) and its restriction to VMWEs unseen in training, computed with the official script for all systems; with the released data our unseen counts differ from the published ones (Appendix A), so unseen F is comparable only within our tables. UMR: AnCast++ (Sun and Xue, 2024) sentence-graph F (F) and the comprehensive score (Comp.); batch and routing runs stop before the document wave and report F. Each configuration was run once; a cold re-run changes F by up to 0.6 in closed mode and by 2.3 for Qwen3.5-9B in fragment mode [E3].

6 Results

6.1 Learning a procedure from 60 sentences

Table 1 gives the PARSEME results. The seed procedures, written before any training sentence was read, already exceed direct generation with ten gold examples (52.4 vs. 39.5 F in Chinese, 44.3 vs. 41.1

System		training sentences	Chinese (1.2) F	Chinese (1.2) unseen F	English (1.1) F	English (1.1) unseen F
Supervised systems, trained on the full training data						
best open track of the shared task		all	71.9	64.2	33.3	23.7
best closed track of the shared task		all	60.9	45.7	32.9	17.6
best published English result (Rohanian et al., 2019)		all	–	–	41.9	–
Direct generation						
Claude Haiku 5.5, 10 gold examples		10	39.5	32.6	41.1	34.8
Decision orchestration (Jev decisions, Sonnet 5.5 reflection)						
seed procedure	Π_0	0	52.4	36.5	44.3	37.8
after 6 rounds	Π_6	30	56.9	43.1	**46.3**	**38.8**
after 12 rounds	Π_{12}	60	57.8	44.3	**46.3**	**38.8**
Π_{12} + routing to Haiku (τ=0.7)		60	**58.5**	**45.7**	44.1	37.4

Table 1: PARSEME test sets: MWE-based F1 and F1 on VMWEs unseen in the training data, computed with the official script from the systems' released predictions (our unseen counts differ from the published ones, §5). Best open track: TRAVIS-mono (Chinese), Milos (English); best closed track: ERMI, TRAPACC. The published English result is quoted from its paper. "Training sentences" are the annotated sentences a system used; the reflector read five per round.

in English), and reflection adds 5.5 F in Chinese (7.8 on unseen expressions) and 2.0 in English. In English the learned procedure is 13 points above the best 2018 shared-task system and 4.4 above the best later result we know of, both trained on 3,471 sentences. Those systems predate current pretrained models, so the comparison shows how little gold the procedure needs, not a settled state of the art. In Chinese, the open-track systems, trained on 35,326 sentences with pretrained encoders, remain 14 points ahead. Precision is the weak side: the final Chinese procedure finds 71% of the gold VMWEs, but only 49% of its predictions are correct. Routing to Haiku re-asked 361 Chinese questions ($0.09) and gained 0.7 F; in English it lost 2.2 F, as Haiku accepted 123 more candidates, 13 of them correct. Whether to route is a question for development data.

6.2 Orchestration versus generating the annotation

Table 2 gives the UMR results. Asked for a whole graph, Sonnet 5.5 reaches 29–33 F; inside the procedure it starts at 45.3 F on T1 and adds the document level (Comp. 44.0 vs. 27.7). For a 9B model, single calls yield no scorable graph on any of 24 documents and a detailed single call reaches 4.9 F on H, while inside the procedure it reaches 17.6, ahead on all 21 documents where both were run. A stronger direct baseline changes the picture for closed decisions. Given the concept inventory and ten gold sentences, Haiku scores 42.0 F on H, above every closed procedure without routing and level with the best one with routing (42.6); in English it scores 46.4 against 30.1 (Table 3). Where the output is mostly graph structure, closed decisions alone do not beat a capable generator given a few examples (§7); a generic tool-using agent is pending [E2].

Who answers the decisions. Very different models can serve the same procedure (Table 7, appendix), but K_0, built with Jev (39.5), loses 8.7–13.9 F with the other decision models, partly through its thresholds (§6.4).

6.3 Learning from a handful of gold annotations

UMR, one document per round. In fragment mode, with Sonnet parsing and Opus reflecting, one training document (51 sentences) raises T1 from 45.3 to 50.8 F and a second raises it to 52.6; temporal relations improve most (29.5 → 43.6), and a third and fourth document leave test scores flat. First parses of new documents agree (Appendix C). DeepSeek gains 5.2 F on T2 reflecting on itself; Qwen3.5-9B does not improve (§7). In closed mode, two rounds of unmeasured reflection raise the Jev procedure on T1 (36.8 → 40.2) and T2 (34.5 → 38.9; 0003 was used for seed tuning). Measured rounds on four more documents (Table 8, appendix) give 39.5 (K_0), 40.9 (K_3) and 40.1 (K_4) on H, while the training documents gain an order of magnitude more; intermediate values await source verification [E13].

System	Decisions by	Refl. docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.	H: 0015–24 F/ Comp.
Baselines: one call per sentence					
Single call	Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9	–
Single call	Qwen3.5-9B	0	–	–	0.0 / 0.0
Detailed single call	Qwen3.5-9B	0	–	–	4.9 / 5.3
Inventory + 10 gold sentences	Haiku 5.5	0 (10)[§]	–	–	42.0 / n.c.
Fragment-mode orchestration (unmeasured reflection by Opus 5.5)					
R_0 (seed)	Sonnet 5.5	0	45.3 / 44.0	–	–
R_{13}	Sonnet 5.5	1 (51)	50.8 / 49.4	–	–
R_{23}	Sonnet 5.5	2 (111)	**52.6 / 51.2**	–	–
R_{32}	Sonnet 5.5	3 (142)	52.6 / 51.1	55.2 / 52.1	–
R_{42}	Sonnet 5.5	4 (201)	n/a	**55.6 / 52.2**	–
R_0 (seed)	DeepSeek V4.1[‡]	0	–	30.8 / 29.0	–
R_{42} (self-reflection)	DeepSeek V4.1[‡]	2 (110)	–	36.0 / 34.0	–
R_0 (seed)	Qwen3.5-9B	0	–	–	17.6 / 18.5
Closed-mode orchestration					
J_0 (seed)	Jev	0[†]	36.8 / 37.1	34.5 / 33.8	34.2 / n.c.
$J_2 = K_0$ (unmeasured reflection)	Jev	2 (111)	40.2 / 41.3	38.9 / 38.0	39.5 / 38.4
K_4 (last reported round)	Jev	6 (168)	–	–	40.1 / 39.2
+ routing to Haiku 5.5, τ=0.7	Jev, Haiku	6 (168)	–	–	**42.6** / n.c.
+ routing to Haiku 5.5, τ=0.9	Jev, Haiku	6 (168)	–	–	41.6 / n.c.
Closed-mode orchestration, measured reflection in 5-sentence batches (Sonnet 5.5)					
J_0 + 10 rounds	Jev	(50)	–	–	35.7 / n.c.
Revised procedure, as released	Jev	2 (111)	–	–	33.3 / n.c.
Π_0: + aspect and polarity conventions	Jev	2 (111)	–	–	38.5 / n.c.
Π_{12}	Jev	2 (111) + (60)	–	–	38.2 / n.c.

Table 2: Main results on Chinese UMR (AnCast++, macro average over documents; F = sentence-graph F, Comp. = comprehensive score including the document level). "Refl. docs" counts automatic-reflection documents only, not all gold exposure. Closed seeds and their descendants additionally inherit manual tuning on 0001–0003 (see §5); round 4 also selects using development scores. "n/a": 0004 was a training document. [†]J_0's rules were tuned by hand on 0001–0003. [‡]Parsed without thinking (Appendix D). [§]Ten gold sentences as in-context examples. "(50)": training sentences read in batches. "n.c.": not computed; these runs stop before the document wave. Red dashes are pending experiments ([E1]); "–" marks a cell not planned; further baselines and K_1/K_2 on H are pending ([E2], [E13]).

Five sentences per round. Table 3 summarizes the batch runs. Twelve rounds cost $0.53–4.27 in reflector calls and at most $0.74 in decisions. On PARSEME, development gains carried over to the test sets. Reflection also adapts a procedure to a new language: the Chinese UMR procedure gains 4.1 F on English, learning conventions such as contracted negation, clitic pronouns, plural nouns and person names. On Chinese UMR, twelve rounds left H unchanged (§7). Of 190 proposed edits, 83 were kept: 62 of 146 rule edits, 9 of 17 threshold changes, 6 of 11 option changes and 6 of 16 rewordings. On PARSEME, 26 of the 37 kept edits change candidate rules, such as a new candidate kind or an exception list (Appendix I).

Run	kept	dev Δ	Π_0	Π_R	$
PARSEME zh	20/34	+7.6	52.4	57.8	0.53
PARSEME en	17/39	+2.9	44.3	46.3	0.69
UMR zh, J_0*	18/32	+7.3	34.2	35.7	2.27
UMR zh, Π_0	15/42	+0.6	38.5	38.2	4.27
UMR en, zh Π_0	13/43	+7.5	26.0	30.1	3.41
UMR en: Π_{12} + routing (τ=0.7)				30.5	
UMR en: Haiku, 10 gold sentences				46.4	

Table 3: Measured reflection in 5-sentence batches (R=12 rounds): edits kept/tried, change in development score, test score of the seed and the last version (PARSEME: MWE-based F; UMR: F on H and the English test set), reflector cost. *Ten rounds, ten development sentences.

6.4 Threshold adaptation and routing

Thresholds do not transfer; retuning is cheap. Answered by the OpenAI Decisions API, K_0 scores 25.6 F on H. Coordinate descent on four gold documents changes two of its ten thresholds and raises F on H, which tuning never saw, to 28.5 (Appendix E). This is task-level threshold adaptation, not calibration of answer probabilities [E4].

Answerer	F	LLM calls	Cost
J_2 on 0010+0012 (20 sentences)			
Jev only	39.1	0	<$0.1
Jev; $c < 0.7$ to Sonnet	42.4	504	≈$9
Sonnet only	43.5	1,292	≈$34
K_4 on H (138 sentences)			
Jev only	40.1	0	$0.75
Jev; $c < 0.7$ to Haiku	42.6	2,554	+$1.18
Jev; $c < 0.9$ to Haiku	41.6	2,988	+$1.47
Haiku, direct generation	42.0	138	$0.19

Table 4: Routing uncertain closed decisions. Within a block, questions and rules are the same in every row; only the answerer of uncertain questions changes. Pilot: public API prices; H: billed costs, routing cost added to Jev's.

Routing. In a 20-sentence pilot, routing questions with confidence below $\tau = 0.7$ from Jev to Sonnet (Table 4) recovers 75% of the F that Sonnet gains by answering everything, at 26% of its cost. On H, routing K_4's uncertain questions to the cheaper Haiku adds 2.5 F at $\tau = 0.7$ but only 1.5 at $\tau = 0.9$, where more questions are routed: Haiku is better than Jev on Jev's most uncertain questions, not on all of them. Jev's confidence predicts agreement with Sonnet (Appendix D), though agreement is not accuracy. The full curve over τ and gold-answered routing are pending [E5].

7 Analysis

Where closed decisions lose on UMR. The answerer matters (with J_2's questions fixed, Sonnet's answers raise F on 0010+0012 from 39.1 to 43.5), but not enough. We replaced parts of an earlier run of K_4 on H (40.4 F) by gold, using the scorer's alignment of predicted to gold nodes. Making every concept form, aspect, role label and attribute of the aligned nodes correct raises F to 49.0; also making the arguments among aligned nodes correct raises it to 55.9. The rest of the gap is nodes the procedure never built. Correct aspect alone is worth 2.9 F, but combining Jev with a lexical prior from the training gold gives 0.68–0.716 aspect accuracy on H, against 0.714 for Jev alone. Attachment is the procedure's own work, and an early routing error moves a whole subtree. On PARSEME, by contrast, the structure is a set of token groups proposed by rules, and every remaining error is a decision or a missing candidate rule, which is exactly what reflection edits.

Options must follow the annotation, not the inventory. The revised UMR procedure completed its inventories from the guidelines, among them the 27 values of the aspect lattice; the training gold uses seven. Given the full lattice, Jev chose values the gold never uses (*directed-achievement* 118 times on H), the judge's aspect score fell from 0.70 for K_4 to 0.31, and F fell to 33.3. The revision also recorded event negation only as modal strength, which the sentence score does not see. Restricting aspect to the seven values and writing negation as `:polarity -` restored aspect to 0.71 and F to 38.5, still 0.9 below J_2, the version it revised. A decision model answers the question it is asked; whether the options are right is a property of the procedure, which only measurement against gold reveals.

Measurement filters edits; small development sets do not. Unmeasured reflection is not filtered: in fragment mode 1 of 441 edits was rejected, and the skills grew from 41k to 156k characters in ten rounds. With measurement, 19 of 48 document-mode and 107 of 190 batch-mode proposals were rejected, among them attempts to undo earlier fixes. Training gains still exceed test gains: in document mode, round 4 raised its development documents by 0.8 F while H fell from 40.9 to 40.1. In batch mode, the size of the development set appears to matter. With 150 development sentences, PARSEME's development gains carried over to the test sets; on Chinese UMR, the run from J_0 with ten development sentences raised development F by 7.3 but H by only 1.6, and the run from Π_0 with 27 accepted fewer edits and moved neither score. The seeds differ, so the comparison is suggestive. Repeated trajectories and matched acceptance-policy ablations remain pending [E3], [E6], [E8].

Weak reflectors and cost. A 9B reflector breaks anchors and writes noisy rules (new documents score 1.4 F below the seed). Closed decisions cost $0.001–0.006 per UMR sentence, against $0.71 in fragment mode with Sonnet (Appendix J) [E11].

8 Conclusion

Decision orchestration learns annotation procedures, not model weights. Where a guideline is a sequence of tests over enumerable candidates, as for verbal multiword expressions, sixty gold sentences and a few dollars give a procedure that beats direct LLM annotation by 5–18 F and, in English, the best published systems we know of. Where the output is mostly structure, as in UMR, orchestration helps small models and enables routing, but

closed decisions alone do not beat a capable generator given examples. Matched baselines, repeated runs and ablations remain necessary.

Limitations

We test two schemes in two languages each, and most configurations were run once. Run-to-run noise (up to 0.6 F for closed mode and 2.3 F for a 9B model in fragment mode) is of the same order as some of the differences we report; repeated runs and other document orders would show the variance of reflection. The UMR test sets are small (99–157 sentences) and differ between stages of the work; unifying them is planned ([E1]). The Chinese set H had been inspected in earlier analyses, and the batch seed Π_0 was derived after inspecting the released revision's scores on it, so all comparisons on H are retrospective. PARSEME English comes from edition 1.1, whose systems predate today's pretrained models; edition 1.2 open-track systems remain far ahead in Chinese, and the second PARSEME language we planned (French) was not run. The PARSEME procedures read the lemmas, parts of speech and dependency relations provided with the data, and our unseen-expression counts differ from the published ones. The seeds were written by the authors, so their quality, and the effort it took to write them, affect every result; we have not measured that effort. Several of the models we use, including all three decision-model services and the frontier LLMs, are commercial APIs whose behavior may change; the release must include shareable predictions, exact versions and permitted cached answers before these runs can be called independently reproducible [TODO: check that the release includes them]. The routing pilot comes from 20 sentences, routing on H was run at two thresholds, and the "human" in routing will be simulated from gold, which overstates real annotators' accuracy. Finally, AnCast and our fine-grained judge depend on an automatic alignment, and the gold itself contains errors. One gold graph has unbalanced brackets and is skipped by the scorer.

Ethics Statement

We use the Chinese UMR annotations supplied in the research checkout and the English annotations of UMR 2.0 (CC BY-SA 4.0). [TODO: Verify the exact release, license, cleaned-gold provenance, redistribution and commercial-API use permissions.] PARSEME 1.2 Chinese is released under CC BY-NC-SA 4.0 and PARSEME 1.1 English under the licenses in its README. We collected no new data from people and no personal information beyond what the released documents contain. Our goal is to reduce the cost of producing the first annotations for a new scheme, which could reduce demand for annotation work. We see the method as a way to direct annotators' effort to the decisions that need human judgment; procedures that annotate cheaply could also produce training data at scale, and their outputs should be checked before they are used as gold. The experiments added in this revision cost about $70 in billed API charges, including pilots and discarded runs; earlier experiments used commercial APIs whose cost and price-date audit is still pending [TODO: sum the API costs].

Generative AI assisted author-side simulated review, manuscript editing, and the implementation and running of the batch-reflection and PARSEME experiments, as described; the authors are responsible for all content. This assistance is not formal ARR peer review.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Julia Bonn, Matthew J. Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benet Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Julia Bonn, Chen Ching-wen, James Andrew Cowell, William Croft, Lukas Denk, Jan Hajič, Kenneth Lai, Martha Palmer, Alexis Palmer, James Pustejovsky, Haibo Sun, Rosa Vallejos Yopán, Jens Van Gysel, Meagan Vigus, Nianwen Xue, and Jin Zhao. 2023. Uniform meaning representation. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of llms as analyzers of abstract meaning representation. *arXiv preprint arXiv:2310.17793*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14953–14962.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696, Online. Association for Computational Linguistics.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *The Eleventh International Conference on Learning Representations*.

Kevin Knight, Bianca Badarau, Laura Baranescu, Claire Bonial, Madalina Bardocz, Kira Griffitt, Ulf Hermjakob, Daniel Marcu, Martha Palmer, Tim O'Gorman, and Nathan Schneider. 2020. Abstract Meaning Representation (AMR) Annotation Release 3.0. LDC2020T02.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1487–1505, Singapore. Association for Computational Linguistics.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2025. Setup: Sentence-level english-to-uniform meaning representation parser. *arXiv preprint arXiv:2512.07068*.

Microsoft. 2026. Use Microsoft-Decision-1 in Azure AI Foundry. https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision. Accessed October 2026.

OpenAI. 2026. Decisions API guide. https://developers.openai.com/api/docs/guides/decisions. Accessed October 2026.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Silvio Ricardo Cordeiro, Agata Savary, Veronika Vincze, Verginica Barbu Mititelu, Archna Bhatia, and 1 others. 2018. Edition 1.1 of the PARSEME shared task on automatic identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Linguistic Annotation, Multiword Expressions and Constructions (LAW-MWE-CxG-2018)*, pages 222–240, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.

Omid Rohanian, Shiva Taslimipoor, Samaneh Kouchaki, Le An Ha, and Ruslan Mitkov. 2019. Bridging the gap: Attending to discontinuity in identification of multiword expressions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2692–2698, Minneapolis, Minnesota. Association for Computational Linguistics.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Manon Scholivet, Agata Savary, Carlos Ramisch, Eric Bilinski, Takuya Nakamura, Maria Mitrofan, and Vasile Pais. 2026. Edition 2.0 of the PARSEME shared task on multilingual identification and paraphrasing of multiword expressions. In *Proceedings of the 22nd Workshop on Multiword Expressions (MWE 2026)*, pages 254–275. Association for Computational Linguistics.

Burr Settles. 2009. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let me speak freely? a study on the impact of format restrictions on large language model performance. In

Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pages 1218–1236, Miami, Florida, US. Association for Computational Linguistics.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-LLM collaborative annotation through effective verification of LLM labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–21, Honolulu, HI, USA. ACM.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*, pages 1–22, New Orleans, LA, USA. ACM.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR.

Jiawei Zhou, Tahira Naseem, Ramón Fernandez Astudillo, Young-Suk Lee, Radu Florian, and Salim Roukos. 2021. Structure-aware fine-tuning of sequence-to-sequence transformers for transition-based AMR parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6290, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A Data, Models and Gold Exposure

Gold revision. The stored AnCast scores in the accompanying rescoring snapshot use the same Chinese gold revision; fine-grained judge analyses still require a separate revision audit and recomputation. Some runs were first scored against an earlier revision; re-scoring every stored prediction moved no macro average by more than 0.3 F. Chinese documents have 8–82 sentences; the development documents 0027–0029 have 47. The English data are the normalized UMR 2.0 documents; we keep the sentences with native annotation, excluding those marked as partial conversions and empty ones. English batches come from the guideline's example sentences (0001–0005) alternating with the Pear story (0580, from sentence 56 on); development is 0580, sentences 31–55; test is 0006–0008, 0578–0579 and 0580, sentences 1–30.

PARSEME. The Chinese test set has 300 VMWEs unseen in train and dev by the official count. With the released evaluation script and data, we count 348 unseen in the training data (and the same number in train and dev), so our unseen F differs from the published values (e.g., 64.2 vs. 60.2 for TRAVIS-mono); global MWE-based F does not depend on this count. The English development set is held out of the training file, since edition 1.1 has no English dev file; the Chinese one is drawn from the official dev file.

Gold exposure. Table 2's reflection counts exclude manual seed tuning. J_0, J_2 and K_0 inherit exposure to 0001–0003 (three unique documents, 142 sentences). K_2 adds 0008–0009 (five unique documents, 162 sentences), and K_4 also adds 0010–0011 (seven, 199). Round 4 additionally uses three development documents (47 sentences) for adaptive selection. These are different uses of gold and must be reported separately; the closed runs are not a two-document-total-data experiment. A batch run reads 60 training sentences (50 for the run from J_0) and measures every trial on its development set: 150 sentences for PARSEME, 27 for Chinese UMR (from 0027–0028; ten in the run from J_0) and 25 for English. The Chinese batch pool consists of five-sentence chunks of 0004–0014 taken round-robin over the documents; it contains T1 and T2, so batch runs are tested on H only. The revised Chinese procedure inherits J_2's exposure, and its Π_0 was derived after the released revision's H score and aspect accuracy had been inspected; the fix follows the training gold's conventions, but the comparison of the two versions on H is retrospective. The seeds' PARSEME candidate rules cover about three quarters of the gold VMWEs of the training data; this coverage was computed once on the Chinese test set as well, without changing the seed.

Models. DeepSeek V4.1 Flash test runs were made without thinking. Qwen models were served with vLLM. The OpenAI Decisions API was used with `gpt-6-luna`; Microsoft-Decision-1 is a post-trained Qwen3.5-9B. In fragment mode the reflector is Claude Opus 5.5 unless the parsing model reflects on itself. Jev in the batch and routing runs is `jev-1.13-20260917`. The batch reflector's temperature is raised to 0.3 when its first reply cannot be parsed.

B Fine-grained judge

The judge works on the AnCast alignment of predicted and gold variables. It scores each of the following checks as an F-score over *units*. A unit is one small comparison (a node, an attribute, an argument set, a relation) that is correct, partially correct, wrong, missing or extra. **(a) nodes**: event found, event concept, entity found, entity concept, granularity (is the node cut at the same words as gold); **(b)** event sense; **(c) attributes**: aspect, modal strength, polarity, refer-person/number, other attributes; **(d) arguments**: core arguments of events, core arguments of abstract predicates, peripheral arguments of events, peripheral arguments of entities; **(e) role labels**: core, peripheral; **(f) names**: name tokens, wiki; **(g)** reentrancy; **(h)** temporal; **(i)** modal; **(j)** coreference chains and subsets; **(k)** document format. For attribution, each unit is assigned to the node that produced its predicted side, or to "no predicted node" when only the gold side exists.

C Training trajectories

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 5: Fragment mode (Sonnet 5.5 parsing, Opus 5.5 reflecting): sentence F on each training document. Iteration y parses with the version before round y, so iteration 1 uses only earlier documents. "after" parses with the document's final version.

Skill	iter 1	iter 2	iter 3	after
`predicate`	–	–	–	–
`np_phrase`	–	–	–	–
`document`	–	–	–	–
(no predicted node)	–	–	–	–
`sent_type`	–	–	–	–
`special_entity`	–	–	–	–
`discourse`	–	–	–	–

Table 6: Credit lost per skill on training document 0001 over three rounds of fragment-mode reflection (sum of $1 -$ credit over all judge units attributed to the skill). Cells are intentionally blank pending current-gold recomputation and scorer-version audit [E13]; this table is a planned diagnostic, not a result.

Answerer	Mode	F	Comp.
Qwen3.5-9B	fragment, R_0	17.6	18.5
Qwen3.8-27B	fragment, R_0	–	–
Sonnet 5.5	fragment, R_0	–	–
Microsoft-Decision-1 (9B)	closed, K_0	30.8	31.6
OpenAI Decisions	closed, K_0	25.6	27.0
+ calibrated (§6.4)	closed, K_0^θ	28.5	29.6
Jev	closed, K_0	39.5	38.4
Sonnet 5.5	closed, K_0	–	–

Table 7: One procedure, different answerers, on H. Pending: [E1], [E5].

Round (doc, sent.)	acc./tried	train F	reg. ΔF	H F
K_0				39.5
K_1 (0008, 10)	8/12	40.0→54.1	+0.5	–
K_2 (0009, 10)	7/12	43.4→60.2	+0.5	–
K_3 (0010, 12)	8/12	41.4→45.9	+0.4	40.9
K_4 (0011, 25)*	6/12	37.5→41.6	+3.2	40.1

Table 8: Reported historical trial summaries for measured reflection with Jev; source-log verification is pending [E13]. "reg. ΔF": mean change on the regression documents. *From round 4 on, problems are diagnosed on the training and regression documents together, and three development documents are measured in every trial (mean +0.8 F on them).

D Additional UMR Results

Who answers the decisions. Because the decision interface is fixed, very different models can serve the same procedure (Table 7). In fragment mode, Qwen3.8-27B reaches 42.4 F on 0001–0004, 94% of Sonnet's 45.1. DeepSeek V4.1 Flash depends on its reasoning: on 0002 it scores 49.1 F with thinking and 35.9 without (Sonnet: 50.6), so its no-thinking test runs in Table 2 understate it. In closed mode the model never writes structure, and a 9B decision service reaches 30.8 F on H, compared with 17.6 for the fragment-mode 9B configuration. This comparison changes both the answerer and the procedure version, so it does not isolate the effect of the closed interface. A procedure tuned for one answerer does not transfer unchanged, though: K_0, built with Jev (39.5), loses 8.7–13.9 F with the other two decision models. Part of the gap is calibration (§6.4).

Thresholds do not transfer; recalibration is cheap. K_0's thresholds were tuned with Jev. Answered by the OpenAI Decisions API, it scores 25.6 F on H. Coordinate descent on four gold documents (0001–0003, 0006) finds improvements for two of the ten thresholds in the tested grid: when a span counts as a single-verb clause (0.6 → 0.7) and the floors for attaching an out-of-clause argument (0.4/0.6 → 0.3/0.5). The calibration objective rises from 29.0 to 30.8, and on H, which calibration never saw, F rises from 25.6 to 28.5 (Comp. 27.0 → 29.6). No value of the other eight improves the objective by more than 0.05 (Appendix E). This grid search found improvements at two thresholds. It is task-level threshold adaptation, not calibration of answer probabilities; the search does not prove that the other thresholds are optimal or causally irrelevant. Calibrating Microsoft-Decision-1, and calibrating from one or two documents, are pending [E4].

Confidence predicts agreement. On J_2's questions for two unseen documents (0010, 0012), Jev and Sonnet agree on 95% of the questions where Jev's confidence is ≥0.9 (53% of questions), 79% at 0.7–0.9 (26%), and 57% below 0.7 (21%). Disagreement concentrates in `sent_type`'s routing question (78%) and `np_phrase`'s modifier questions (67%), where a wrong answer reshapes a subtree. Agreement with Sonnet is not accuracy against gold or calibrated confidence. Reliability against gold-derived answers is pending [E4].

E Task-Level Threshold Adaptation

Table 9 lists the thresholds of the closed-mode procedure and the sweep for the OpenAI Decisions API. The objective is Eq. 1 on 0001–0003 and 0006. Thresholds were swept in the order listed, each on top of the values kept so far, and a value is kept if it improves the objective by more than 0.05.

F Skills

A fragment-mode skill. `predicate/skill.md` has sections on the head concept (event, abstract

Threshold (skill)	Jev	tried	kept (ΔJ)
comma span → discourse (`sent_type`)	.2	.3 .4 .5	.2
split noun phrase (`sent_type`)	.2	.3 .4 .5	.2
word is an event (`sent_type`)	.5	.4 .6 .7	.5
single-verb clause (`sent_type`)	.6	.4 .5 .7	**.7** (+1.55)
conjunct cut, and/or (`discourse`)	.4/.5	+.1, +.2, +.3	.4/.5
cut without comma (`discourse`)	.5	.6 .7 .8	.5
outside argument, ARG0-1/other (`predicate`)	.4/.6	±.1, +.2	**.3/.5** (+0.27)
generic yes/no (shared)	.5	.6 .7 .8	.5
coreference kept (`document`)	.4	.3 .5 .6	.4
modal source kept (`document`)	.7	.5 .6 .8	.7

Table 9: Calibration of K_0 for the OpenAI Decisions API. Objective 29.0 → 30.8 on the calibration documents; on H, F 25.6 → 28.5. [TODO: same table for Microsoft-Decision-1 (E4)]

Node	Kind	Decides or does
`sent_type`	decision	span type (5-way); routing
`discourse`	decision	relation; split points
`predicate`	decision	head, sense, roles, aspect, modality
`np_phrase`	decision	head, its form, modifier roles
`special_entity`	decision	entity kind, type, value
`document`	decision	time anchor, source, coreference
`antecedent`	rule	out-of-span participants
`coverage`	rule	re-dispatch uncovered tokens
`reentrancy`	rule	merge same-referent nodes
output handler	rule	assemble, validate

Table 10: Nodes of the UMR procedure. Each decision node has two interchangeable implementations (fragment and closed); rule nodes, registry and scorer are shared.

predicate, sense), the participants, participants outside the clause, aspect, modal strength and polarity, the output, worked examples, and, last, an `## Input` section with the span, the numbered sentence and the outputs of the parent node. Because the input comes last, everything before it is cached across calls. Reflection edits these sections in place by anchored edits. The emit schema lists closed value sets for aspect (27 values), modal strength (6) and polarity, and open fields for the fragment, its open slots and its out-of-span participants. Over ten rounds of unmeasured reflection, the six decision skills grow from 41k to 156k characters of instructions.

A closed-mode skill. A closed skill is a Python generator:

```
def run(sent, span, **_):
    a = yield {"state": {...}, "questions": {
        "type": {"type": "choice",
                 "criteria": TYPES}}}
    b = yield {...}   # may depend on a
    return emit_from(a, b)  # rules
```

Each option in `TYPES` carries a description condensed from the guideline, for example what counts as a *discourse* span in Chinese, with the constructions that do and do not qualify. The returned emit has the same schema as the fragment-mode skill, so the registry and rule nodes are unchanged. Table 11 lists the questions per skill.

Skill	Rounds of closed questions
`sent_type`	span type (5-way); for a one-word span in PropBank, its use here (event, state, manner, thing)
`discourse`	relation (8 frequent ones); one yes/no per possible cut point
`predicate`	(1) head word, verb or abstract predicate; (2) sense, aspect, modal strength, negation; (3) role of each content word; antecedent of each core argument outside the clause
`np_phrase`	(1) head and its form (8-way); (2) role of each modifier; sense if the head is an event
`special_entity`	kind (name, date, quantity, percentage, ordinal); entity type (44); quantity type
`document`	(1) temporal relation to the anchor event; source of a view; earlier mention it corefers with, or new; (2) confirm each coreference (yes/no)

Table 11: Closed questions asked by each decision skill of J_0. Values (numbers, dates, names) are read by rules.

G Reflection prompts

Algorithm 1 Measured reflection in small batches.

Require: seed procedure Π_0, training pool P, development set D, batch size b=5

for $r = 1, \ldots, R$ **do**
  $B \leftarrow$ next b sentences of P; $G \leftarrow$ batches of rounds $r-2, r-1$
  $s \leftarrow \textsc{Score}(\Pi, \{B, G, D\})$
  $E \leftarrow \textsc{Reflect}(\Pi, \textsc{Report}(\Pi, B))$ ▷ ≤ 3 edits
  for $e \in E \cup \textsc{Revise}(\text{rejected})$ **do**
    $s' \leftarrow \textsc{Score}(e(\Pi), \{B, G, D\})$
    if $\bar{s}' - \bar{s} \geq 0.3 \wedge s'_D - s_D \geq -0.5 \wedge s'_G - s_G \geq -1$ **then**
      $\Pi \leftarrow e(\Pi)$; $s \leftarrow s'$
    end if
  end for
end for

Batch mode. The system prompt describes the procedure (skills that split the input by rules, ask Jev closed questions and assemble the answers by rules) and ranks what makes a good edit: the correct answer must be among the options and a rule must turn it into the gold structure; a closed question is better than a rule that guesses; long or ambiguous option lists should be split or given a *none* option; thresholds are constants to change; gold conventions belong in rules; rewording only when an option is clearly misread. Edits must be general

(no special-casing of a sentence from the report), keep the code runnable, and not depend on each other. The user prompt contains the task description, the full text of the editable files, the titles of edits tried in earlier rounds with their outcome, and the batch report: for each sentence, the gold and predicted annotations, the scorer's findings, and for every node the questions asked with Jev's top options and probabilities. The reflector answers in JSON with a diagnosis and at most three edits. Rejected edits are returned once with their measured effect on each set and the reason for rejection.

Document mode. [TODO: condense rsi/find_problems.md, rsi/plan_edits.md and rsi-jev2/diagnose.md into half a page each; include the list of proposal kinds and the acceptance constants of §4]

H The PARSEME Procedures

Candidates. Chinese: (1) a verb, auxiliary, adjective or noun token of two or more Han characters ending in a directional, resultative or co-verb complement from a fixed list (上, 下, 出, 起, 開, 完, 好, 掉, 住, 到, 為, 成, 在, 入, 至, 給, ...), read as stem + complement; (2) a four-character Han token; (3) a verb and a complement token attached to it (`mark`, `compound:vv`, ...) within three tokens, or adjacent to it; (4) a light or causative verb from a list of 60 (進行, 提供, 作出, 受到, 帶來, 造成, 引起, ...) and its object. English: (1) a verb and its particle (`compound:prt`, or a particle word as `advmod` within three tokens); (2) a light verb from a list of 45 (*make, take, give, have, do, pay,* ...) and its object noun; (3) a verb and the preposition right after it; (4) *make, let, get, break, come, go* with a bare complement.

Questions. One choice per candidate, with options by kind. Complement candidates (Chinese): VPC.semi, MVC, none. Light-verb candidates: LVC.full, LVC.cause, VID, none. Four-character tokens: a yes/no question ("a fixed idiom used as a verb or predicate here"). Particle candidates (English): VPC.full, VPC.semi, none; preposition candidates: IAV, VPC.full, none; complement candidates: VID, none. Every option carries one or two sentences from the guideline with examples. A candidate is kept when its most probable option is a category with $p \geq 0.5$; overlapping candidates are resolved by probability.

After reflection. The Chinese procedure grew from 64 to 80 lines of candidate rules and from 78 to 95 lines of decision code; the English one from 44 to 83 and from 69 to 96. New rules include exception lists (complements after perception verbs, causatives followed by a clause), new candidate kinds (English *verb* + *particle* + *of*, idiom candidates *verb* + *preposition* + *noun*), and per-verb thresholds (a stricter one for *get* + noun).

Why PARSEME. The PARSEME guidelines are, almost literally, a decision procedure. Candidates are found on the dependency trees that ship with the data, and each candidate is classified by linguistic tests organized as decision trees into a closed set of categories (verbal idioms, light-verb constructions, inherently reflexive verbs, verb-particle and multi-verb constructions). The test sets contain about 300 expressions per language that are unseen in training, which is the regime a procedure learned from a few documents should help with. The data is freely available, has an official scorer, and includes Chinese, which keeps the language constant with our UMR experiments. We added English from edition 1.1, the last edition with English, instead of the French or Polish we had planned, so that both benchmarks share their two languages. Edition 2.0 extends the scheme to all syntactic categories in 17 languages (Scholivet et al., 2026), which allows a second test after a schema revision.

Controls not run. The plan also called for a direct Jev classifier that selects a category or *none* for each candidate with the same frozen candidate generator and assembler, a Sonnet guideline/few-shot system, and a fragment-mode variant. They were not run; the direct baseline we report uses Haiku with ten gold examples [E9].

I Examples of Kept Edits

Each edit is an exact find-and-replace in one file of the procedure, proposed by the reflector with a reason and kept because its measured effect met the criterion of §4. Gains are F on the batch, the regression set and the development set.

PARSEME English, round 6 (`candidates.py`; batch +14.3, regression ±0, dev +1.9). *Reason given:* "*V as NP* marks a role or function (*acts as, serve as, work as*) and is not an IAV; removing it avoids false positives of this kind generally."

```
- ... not in PARTICLES | {"to"}
+ ... not in PARTICLES | {"to", "as"}
```

PARSEME Chinese, round 3 (`candidates.py`; batch +7.3, regression ±0, dev +0.7). *Reason given:* "A causative 讓/使/令, or 有, followed by a verb or adjective is a clausal complement, not a light verb plus event noun." The light-verb candidate rule gains the condition `and not (h.form in {讓, 让, 使, 令, 有} and t.upos != NOUN)`.

UMR English, round 9 (`jevlib.py`; batch +6.1, regression ±0, dev +1.1). *Reason given:* "Contracted pronouns such as *he's*, *you're* and *they'll* are recognized as pronouns. They then get a person node with `:refer-person` and `:refer-number` instead of a literal concept like `he's`."

```
+ elif lang == "en":
+   for s in ("'s","'ll","'d","'m","'re","'ve"):
+     if (t.endswith(s) and
+         t[:-len(s)] in PRONOUNS["en"]):
+       t = t[:-len(s)]
+       break
```

J Runs and Costs

Item	requests	$
Reflection (Sonnet requests; $ Sonnet / Jev)		
PARSEME zh	18	0.53 / 0.01
PARSEME en	22	0.69 / 0.01
UMR zh (Π_0)	23	4.27 / 0.74
UMR en	22	3.41 / 0.24
Test sets		
PARSEME zh test, Π_0, Jev	1,448	0.06
PARSEME zh test, cascade	361	0.09
PARSEME en test, cascade	720	0.14
UMR zh test, K4, Jev	6,500	0.75
UMR zh test, K4, cascade	2,554	1.18
Direct, PARSEME zh / en	3,462 / 3,965	1.16 / 0.85
Direct, UMR zh / en	138 / 143	0.19 / 0.11

Table 12: Billed costs. Decision costs during reflection are low because unchanged questions are answered from the cache.

K Pending experiments (to be removed before submission)

E1 One gold revision, one held-out set (H), one development set for every system in Table 2.

E2 Stronger baselines: full guideline with gold examples from the same training documents; a generic tool-using agent; Chun and Xue (2024). *Partly done:* inventory + ten gold sentences with Haiku 5.5 on H, the English UMR test set and both PARSEME test sets.

E3 Learning curves over 0/1/2/4/8 documents with three document orders.

E4 Calibration: reliability diagrams and ECE against gold-derived answers for three decision models; calibrating Microsoft-Decision-1; calibration from 1, 2 and 4 documents; transfer of thresholds between models.

E5 Routing: the full curve over τ on H; routed questions answered from gold as a simulated annotator; Sonnet as the only closed answerer on H. *Partly done:* K_4 routed to Haiku at $\tau \in \{0.7, 0.9\}$ on H; PARSEME at τ=0.7.

E6 Measured reflection: ablation of round 4, a larger and more varied development set, rounds 5–7, and the same loop with the other two decision models.

E7 Ablations of the procedure: without the coverage and reentrancy rules; whole-graph generation instead of fragments; without the PropBank tool.

E8 Ablations of reflection: wording-only edits vs. edits to rules, options and structure; unmeasured vs. measured acceptance at equal budget; training-only vs. training + regression + development measurement.

E9 Second benchmark. *Done* for PARSEME 1.2 Chinese and 1.1 English with batch reflection (Appendix H); the direct Jev classifier, Sonnet few-shot and fragment-mode controls, threshold tuning and learning curves over 1/2/4 batches of 100 sentences are not run.

E10 English UMR with the same procedure. *Done* with batch reflection from the Chinese Π_0 (Table 3).

E11 Cost table: calls, tokens, dollars and wall-clock per configuration. *Done* for the batch, routing and direct runs (Table 12); pending for the earlier runs.

E12 Annotator time: correcting the procedure's output vs. annotating from scratch.

E13 Evidence audit: recompute fine-grained judge results on the current gold; locate original K_1/K_2 predictions and scores; record scorer and model versions, gold checksums, invalid-graph handling, and the exact copying-guard coverage.

Planned generalization check. To distinguish useful supervised reflection from fitting the diagnosis documents, we will compare the seed, the current reflection protocol and a cross-document

selection protocol under matched gold, reflector and proposal budgets. The latter will propose edits from recurring training errors, state their guideline basis and applicability, and evaluate them on training-pool documents excluded from that proposal's diagnosis. Regression effects, instruction growth and cost will also be recorded. These selection documents are not final-test data. Checkpoint selection will use development data under a fixed query budget; frozen versions will be evaluated with repeated runs on a separate final set. The previously inspected H trajectory remains retrospective evidence, rather than a fresh confirmation set [E3], [E6], [E8].

L Reproduction Status

The bundled AnCast rescoring JSON supports several reported macro averages but does not include the intermediate K_1/K_2 entries. Those H-score cells and current-gold judge counts remain blank [E13]. The rescoring notes identify predictions from a separate experimental branch and a scorer not bundled with this checkout; the main-branch scorer also has an unresolved missing-variable path for failed graph matching. Before reproduction, provide a frozen scorer, all relevant predictions, gold checksums and exclusions, code commits, service/model versions and dates, runtime settings and a complete data-exposure ledger. Do not regenerate result tables with a different gold or silently drop failed predictions. Current cached-answer availability and redistribution permissions remain to be verified; commercial API access is not needed to audit already stored predictions.

The batch, routing and direct-generation runs of this revision are complete in the code repository: `rsi-batch/` holds the driver and the two task adapters, and `experiment/decision-orchestration/` every version Π_r with each round's report, proposals, trials and decision, the test-set predictions, request logs with billed costs, and `summary.md` listing every number. A run is started with `rsi-batch/run_parseme.sh LANG 12` or `rsi-batch/run_umr_v11.sh LANG 12` and tested with `rsi-batch/evaluate_test.py`.